Chapitre 4 : Polymorphisme et liaison dynamique

Cette fiche fait le point sur le concept de polymorphisme en POO/Java et sur le mécanisme de résolution d'appel de méthodes (partie statique et « liaison dynamique ») en Java.

Polymorphisme ?

Une notion essentielle

Le polymorphisme constitue la troisième caractéristique essentielle d'un langage orienté objet après l'abstraction des données (encapsulation) et l'héritage.

Bruce Eckel, *Thinking in JAVA*.

Pour introduire le polymorphisme (« *qui peut prendre plusieurs formes* »), considérons l'exemple (désormais célèbre ?) du diagramme UML de la figure suivante :

Voici un code possible pour certaines des classes de notre hiérarchie :

```
public class Animal {
    private String nom;

    public Animal(String nom) {
        this.nom = nom;
    }

    public String getNom() {
        return nom;
    }

    public void setNom(String nom) {
        this.nom = nom;
    }

    public void crier() {
        System.out.println("Je ne sais pas comment crier !");
    }

    public void crier(String raison) {
        System.out.print("Je vais crier car " + raison);
        crier();
    }
```

```
    public String toString() {
        return "Mon nom est " + nom + ". Je suis un Animal ";
    }
}
public class Prof extends Animal {
    private int nbEtudiants;

    public Prof(String nom, int nbEtudiants) {
       super(nom);
       setNbEtudiants(nbEtudiants);
    }

    public void setNbEtudiants(int nbEtudiants) {
        if(nbEtudiants < 0) {
            throw new IllegalArgumentException("nbEtudiants négatif !");
        }
    }

    @Override
    public void crier() {
        System.out.println("Grrrr !");
    }

    @Override
    public String toString() {
        return super.toString() + "de type Prof. " + " J'ai " + nbEtudiants
+ " étudiants";
    }
}
public class Pangolin extends Animal {
    ...
    @Override
    public void crier() {
        System.out.println("Gwwark Rhââgn Bwwikk ");
    }

    @Override
    public String toString() {
        return super.toString() + "de type Pangolin avec " +
getNbEcailles() + " écailles.";
    }
    ...
}
public class PangolinALongueQueue extends Pangolin {
    private int longueurQueue;

    public PangolinALongueQueue(String nom, int nbEcailles, int
longueurQueue) {
        super(nom, nbEcailles);
        this.longueurQueue = longueurQueue;
    }

    @Override
    public crier() {
        System.out.print("Qeyyyoouuuu "); // partie spécifique
        super.crier();                    // partie commune (réutilisation)
    }

    @Override
    public String toString() {
        return super.toString()
            + "\nMais je suis aussi un Pangolin spécial : en plus, j'ai une
queue de "
            + longueurQueue + " cm !";
    }
}
```

Polymorphisme de méthodes

Cet exemple comprend une première catégorie de polymorphisme, que vous connaissez déjà en fait : le *polymorphisme de méthodes*, lié à la surcharge et la redéfinition :

- **Surcharge :** la méthode `void crier(...)` est surchargée dans la classe `Animal`. Il en existe donc deux versions (deux « formes ») avec des signatures différentes dans cette classe (et donc aussi les classes qui héritent de `Animal`).
- **Redéfinition :** la méthode `void crier()` (sans paramètre) est redéfinie dans la plupart des sous-classes, pour tenir compte du fait que chaque animal a un cri spécifique. Elle existe donc en plusieurs versions (plusieurs « formes », mais avec cette fois ci la même signature) réparties dans la hiérarchie de classe.

Polymorphisme d'objets

Cet exemple permet de plus d'introduire une deuxième catégorie de polymorphisme, parfois appelé *polymorphisme d'objets*.

Rappel : la relation d'héritage entre les classes `Animal` et, par exemple, `Prof` traduit en Java le fait que, « dans la vraie vie », un prof *est-un-type* particulier d'animal (si, si...).

Poussons un peu plus loin. Puisque, grâce à l'héritage, un objet instance de la classe `Prof` « est » un type particulier d'`Animal`, ne serait-il pas possible en Java de manipuler un objet `Prof` « en tant que » `Animal` ? Eh bien oui, tout simplement au moyen d'une référence de type `Animal`. Voyons ce que cela donne :

```
public class Test {
    public static void main(String args[]) {
      Prof p = new Prof("Sidonie", 1542);
         // La classe Prof dérive de la classe Animal.
         // => Un objet Prof "est" aussi un Animal
         // => On peut manipuler un objet de type Prof "en tant que" Animal
!
         Animal unProfEstUnAnimal = p; // et voila !

         // Notez qu'on pourrait même écrire directement sans problème :
         // Animal unProfEstUnAnimal = new Prof("Sidonie", 1542);

         // Désormais, unProfEstUnAnimal est une référence de type Animal
         // qui manipule (réfère) un objet dont le "vrai" type est Prof !

         // Faisons crier notre Animal... qui est un Prof... donc fait
crier le prof. Aie !
         unProfEstUnAnimal.crier(); // affiche "Grrrr"

         // affiche l'état du Prof :
         // "Mon nom est Sidonie. Je suis un Animal de type Prof. J'ai 1542
étudiants"
         System.out.println(unProfEstUnAnimal.toString());
    }
}
```

De cet exemple, on retiendra que :

Avec le polymorphisme, on peut toujours manipuler une instance d'une classe fille au moyen d'une référence du type de n'importe quelle super classe.

Plus fort, le polymorphisme permet par exemple de stocker (... manipuler) ensemble plusieurs objets de types différents si leurs classes héritent toutes d'une même classe mère, au moyen par exemple d'un tableau du type de la classe mère.

Il permet encore d'écrire des traitements (des méthodes) génériques, qui s'appliqueront indifféremment à des objets quelque soit leur type réel, pour peu que leurs classes héritent toutes de la même classe mère.

Voyons tout cela par l'exemple :

```
public class Test {
    public static void main(String args[]) {
        // On peut stocker dans un tableau
        // un mélange de Prof et de Canard... Et même des pangolins à
longue queue
        Animal tab[] = new Animal[10];
        tab[0] = new Prof("Pierre", 12);
        tab[1] = new Canard("Paul", 122);
        tab[2] = new PangolinALongueQueue("Jacques", 765, 42);
        ...

        system.out.println(tabFig[2]);
        // Affiche :
        // Je m'appelle Jacques. Je suis un Animal de type Pangolin avec
765 écailles.
        // Mais je suis aussi un pangolin spécial : en plus, j'ai une queue
de 42 cm !";


        // On peut appliquer un traitement générique à tous les animaux
        // du tableau. Peu importe qu'ils soient des Pangolin ou des
Diplodocus !
        ajouterANom(tabFig[0], "Kiroule");
        ajouterANom(tabFig[1], "Yesterre");
        ajouterANom(tabFig[2], "Kadi");


        system.out.println(tabFig[2].getNom());
        // Affiche :
        // Jacques Kadi.

        faireCrierUneMenagerie(tabFig, "Oula, que de bruit ! : ");
        // Affiche :
        // "Oula, que de bruit !
        //  Grrr
        //  Coin coin
        //  Qeyyyoouuuu Gwwark Rhââgn Bwwikk"
    }


    // On peut écrire des traitements génériques
    // qui s'appliqueront à des objets de types divers !
    public static void ajouterANom(Animal a, String complement) {
        String nomComplet = a.getNom() + complement;
        a.setNom(nomComplet);
    }

    system.out.println(tabFig[2]);
```

```
    // Affiche :
    // « Je m'appelle Jacques Kadi. Je suis un Animal de type Pangolin avec
765 écailles.
    // Mais je suis aussi un pangolin spécial : en plus, j'ai une queue de
42 cm ! »;

    // On peut même écrire des traitements génériques
    // sur des conteneurs (ici, un tableau)
    // qui contiennent des références sur
    // des objets de types différents !
    public static void faireCrierUneMenagerie(Animal tab[], String
commentaire) {
        System.out.println(commentaire);
        for(Animal a: tab) {
              a.crier() ;
        }
        // Notez que, suivant le type de l'objet référencé par "a"
        // (Prof, Pangolin, PangolinALongueQueue ou Canard)
        // ce ne sera en fait pas la même méthode crier()
        // qui sera in fine exécutée : chaque animal criera
        // à sa manière, précisée dans chaque redéfinition de la méthode
crier() !
    }
}
```

De cet exemple, on retiendra le principe fondamental suivant :

Principe de substitution

Avec le polymorphisme, une instance d'une classe fille peut être utilisée partout où une référence du type de sa classe parent est attendue.

Type statique, type dynamique

Le polymorphisme d'objet conduit à distinguer en POO deux notions de type pour chaque objet :

- le **type dynamique** de l'objet, c'est à dire le type effectif de l'objet en mémoire. Il est déterminé une fois pour toutes au moment de la création de l'objet avec `new`.
- le **type statique** de la référence qui, à un endroit du code, est utilisé pour manipuler cet objet. Le type statique peut varier au cours du programme, en fonction du type des références qui, au fur et à mesure, sont utilisées pour manipuler l'objet.

Dans le premier exemple ci-dessus, il n'existait qu'un seul objet en mémoire, de type (dynamique) `Prof`. Mais cet unique objet est ensuite manipulé au moyen de deux références (types statiques) : la référence `p` de type `Prof`, puis la référence `unProfEstUnAnimal` de type `Animal`.

Avec les notions de type statique et de type dynamique, on peut maintenant donner une définition plus précise du polymorphisme d'objets.

Polymorphisme d'objets

Le polymorphisme d'objets dénote le fait que le type statique d'un objet peut être n'importe quel type *plus générique* que le type dynamique de cet objet :

- n'importe quelle classe située *au dessus* de la classe du type dynamique de l'objet dans la hiérarchie d'héritage (la classe mère de la classe de l'objet, la classe mère de cette classe mère, et ainsi de suite...) ;
- ou n'importe quelle interface implantée par la classe du type dynamique de l'objet (pas d'impatience : vous apprendrez ce qu'est une interface dans la prochaine fiche...).

Interroger le type dynamique durant l'exécution du programme

Le compilateur Java connaît toujours les types statiques (les types des références) utilisés dans le code. Par contre, il n'a pas connaissance du type dynamique de l'objet référencé.

Plusieurs mécanismes existent toutefois pour consulter le type dynamique d'un objet durant l'exécution du programme. Voici, par l'exemple, les plus courants :

```
public class Test {
    public void exempleDeMecanismesTestantLeTypeDynamique(Object o) {
      // o est une référence (type statique Object)
      // sur un objet de type dynamique quelconque...
      // L'objet qui est "au bout" de la référence o
      // peut être un Animal, un Pangolin (avec ou sans longue queue)
      // une instance de la classe AvionEnPapier,
      // une instance d'Object ou tout autre chose...
      // Allez savoir...

      // Il est possible de récupérer une chaîne de caractères contenant
      // "en toutes lettres" le type dynamique d'un objet :
      System.out.println(o.getClass().getName() ) ;

      // Test sur le type dynamique de l'objet :
      if (o.getClass() == Pangolin.class) {
        // VRAI si et seulement si le type dynamique
        // de o est Pangolin
        ...
      }

      // Vérification de la cohérence entre type avec l'opérateur
instanceof
      if (o instanceof Animal) {
        // VRAI si et seulement si
        // le type dynamique de o est Animal
        // OU n'importe quel sous classe de Animal

       // Donc vrai par exemple si o réfère un objet de type
       // dynamique Prof
        ...
      }
   }
}
```

Attention toutefois à l'usage de ces mécanismes d'interrogation du type dynamique ! Dans de rares cas, comme celui de la méthode `boolean equals(Object o)` discuté plus loin dans cette fiche, ils sont indispensables. Il faut donc les connaître. Par contre, hormis ces cas, **leur usage est souvent le signe d'une mauvaise conception objet**.

Un bon développeur POO n'a (presque...) jamais besoin d'interroger le type dynamique de l'objet qui se cache « au bout » de la référence qu'on utilise pour le manipuler !

Conversion (transtypage, coercition, *cast*) de type statique entre classes

Transtyper (convertir, *caster*) une référence vers un type plus générique est toujours possible. Cela s'appelle un *upcast* en bon français, ou coercition ascendante. On peut toujours écrire, par exemple :

```
Pangolin p = new PangolinALongueQueue("Tartiflette", 12, 134);
Animal   a = p;  // ou upcast explicite: Animal a = (Animal) p
Object   o = p ; // ca marche aussi, car toute classe dérive de Object en Java !

System.out.println(o.toString()) ;
// et il s'affiche toutes les propriétés du Pangolin, y compris la longueur de son appendice !
```

A l'inverse, transtyper une référence vers un sous type (ce qu'on appelle un *downcast* en mauvais anglais, ou coercition descendante) appelle quelques précautions :

- A la compilation, cela nécessite un *cast* explicite dans le code, sans quoi le compilateur provoque une erreur.
- A l'exécution, cela nécessite que le type statique cible soit compatible avec le type dynamique de l'objet. Autrement dit, il faut que le type statique cible soit plus générique que le type dynamique de l'objet. Si ce n'est pas le cas, la machine virtuelle lève une exception de type `ClassCastException`.

Tout ceci est expliqué dans les commentaires de l'exemple suivant :

```
public class Test {
    public static void main(String args[]) {
        Animal uneBebete;

        switch (new Random().nextInt(2)) { // 0 ou 1, alétoirement
        case 0:
            // un canard à 2 plumes, pauvre bête...
            uneBebete = new Canard("Coin coin", 2);
            break;
        case 1:
            // un prof sans élève... You know what ? Is he Happy ??
            uneBebete = new Prof("Droopy", 0);
            break;
        }
        // Désormais, uneBebete réfère aléatoirement un Canard ou un Prof

        // Affichons l'état notre animal (que ce soit un Canard ou un Prof)
        System.out.println(uneBebete);

        // La ligne suivante est refusée à la COMPILATION car
        // Potiron n'est (sans doute... ???) ni une super classe,
        // ni une sous classe de Animal, mais une classe
        // dans une hiérarchie de classe tout à fait différente...
        Potiron uneCucurbitacee = (Potiron) uneBebete;

        // La ligne suivante est refusée à la COMPILATION.
        // Canard est une sous classe de Animal.
        // Il s'agit donc d'un cast vers un sous type ("downcast").
        Canard c = uneBebete;
```

```
        // Eh oui. Pour un downcast, Java impose qu'on écrive
        // un cast explicite dans la code.

        // Voila comment on fait.
        // La ligne suivante est acceptée à la COMPILATION :
        Canard c = (Canard) uneBebete;

        // Lors de l'EXECUTION de la ligne précédente,
        // si jamais uneBebete réfère un objet de type dynamique Prof,
        // alors il vaudrait mieux que Java le détecte...
        // Eh oui, un prof est certes un animal, mais tout de même pas un
canard !

        // Fort heureusement, la machine virtuelle va s'assurer que ce cast
est possible.
        // De deux choses l'une :
        // - Si le type dynamique de uneBebete est Canard (ou une sous
classe de Canard),
        //   alors tout se passe bien et l'exécution continue.
        // - Si le type dynamique de uneBebete est Prof (ou Pangolin,
ou...),
        //   alors la machine virtuelle lève une exception de type
ClassCastException.
        //   Et comme dans notre cas cette exception n'est récupérée nulle
part,
        //   le programme va s'arrêter immédiatement...

        c.setNbPlume(2222); // voila notre canard (si c'en est bien un...)
bien remplumé !

        // Avec instanceof, on peut s'assurer de la validité du downcast
avant de l'exécuter :
        if(anim instanceof Canard) {
                // avec le test qui précède, le cast qui suit ne lèvera
jamais d'exception :
                Canard c = (Canard) anim;
                ...
        }

      }
}
```

Attention toutefois aux usages du *downcast*. Il faut savoir le faire mais il est souvent le signe d'une mauvaise conception objet.

Un bon développeur POO ne devrait (presque...) jamais avoir besoin de transtyper une référence vers un sous-type (*downcast*).

Mécanisme d'appel de méthode en Java. Liaison dynamique

Jusqu'ici, à chaque fois qu'on a fait crier un animal quelconque (par exemple un `PangolinALongueQueue`), même si c'est avec une référence de type `Animal`, la pauvre bestiole a toujours crié comme il faut (par exemple comme tous ses amis à longue queue) :

```
Animal a = new PangolinALongueQueue("Kirikou", 12, 120);
a.crier(); // affiche le cri des Pangolins à longue queue, même si a est de
type Animal

System.out.println(a.toString());
// affiche :
// "Je m'appelle Kirikou. Je suis un Animal de type Pangolin avec 12
écailles.
// Mais je suis aussi un Pangolin spécial : en plus, j'ai une queue de 120
cm !"
```

C'est un peu magique tout ça, non ? Faisons le point...

Supposons qu'on exécute une méthode au moyen d'une référence de type donné (type statique) qui réfère un objet de type dynamique donné. Cette méthode peut avoir été *surchargée*, dans la classe du type dynamique de l'objet, ou dans une de ses super-classes. Elle peut aussi avoir été *redéfinie* à différents niveaux de la hiérarchie. **Comment Java sélectionne-t-il la méthode qui sera finalement exécutée ?**

Une manière pas trop bête de se représenter intuitivement ces mécanismes est la suivante.

Le choix de la méthode dépend du type statique et du type dynamique de l'objet, ainsi que des types statiques des paramètres. Il est fait en deux temps :

- **À la compilation, le compilateur détermine la signature précise de la méthode qui sera exécutée**, parmi toutes les surcharges. Il ne connait pour cela que les types **statiques** de l'objet sur lequel la méthode est appelée et des paramètres passés à la méthode. Il dresse la liste des signatures de toutes les méthodes connues dans la classe du type **statique** (y compris les éventuelles méthodes héritées), puis il choisit celle dont la signature est « la plus adaptée » (on dit la **plus spécifique**) aux types statiques des paramètres effectifs utilisés lors de l'appel.

  On résume parfois cette étape en disant que **la surcharge est résolue à la compilation** en fonction des types statiques.

- **À l'exécution la machine virtuelle exécute la méthode *de l'objet* qui a la bonne signature**. Parmi toutes les redéfinitions situées au dessus du type dynamique de l'objet, ce sera celle la plus proche du type dynamique de l'objet. Ce mécanisme s'appelle **la liaison dynamique**.

  Pour l'intuiter, rappelons que, en POO, l'exécution d'une méthode peut-être vue comme l'« envoi d'un message » à un objet : c'est *l'objet* lui-même qui est chargé d'exécuter *sa* méthode, connaissant sa signature. Si la méthode a été redéfinie dans la hiérarchie de classe, la version de la méthode que possède l'objet est celle qui est la plus proche de la classe de notre objet, en remontant la hiérarchie des classes. C'est cette méthode qui sera exécutée.

  On résume parfois cela en disant que **la rédéfinition est résolue à l'exécution** en fonction du type dynamique de l'objet sur lequel on invoque la méthode.

Dans l'exemple qui précède, lorsqu'on a exécuté `a.crier()` :

- le compilateur a sélectionné la signature `void crier()` sans paramètre ; il a éliminé la surcharge `void crier(String raison)`.
- la machine virtuelle a exécuté la méthode `void crier()` de l'objet. L'objet est de type `PangolinALongueQueue`. Comme cette classe a redéfini la méthode `void crier()`, c'est la version de cette classe qui était, *in fine* exécutée.

Pour ceux d'entre vous qui veulent en savoir plus, voici une explication détaillée du mécanisme d'appel de méthode en Java.

En savoir plus

Pour détailler ce mécanisme, on s'appuiera sur le pseudo-code suivant :

```
// TypeStatique est une super-classe de TypeDynamique
TypeStatique ref = new TypeDynamique(...);
<type_retour> tmp = ref.nomDeMethode($param1$, $param2$, ..., $paramN$ );
```

Traitement à la compilation (partie statique)

Rappel: Lorsqu'un objet est « sur-classé », le compilateur ne connaît que le type de la référence utilisée pour le manipuler. Il ne connaît pas les types dynamiques des objets référencés.

Le traitement à la compilation repose sur les principes suivants :

- Le compilateur récupère le type statique de la référence sur laquelle la méthode est appelée (dans notre cas : `TypeStatique`).
- Le compilateur liste toutes les surcharges nommées `nomDeMethode` que possèdent la classe `TypeStatique` : celles définies dans la classe elle-même, mais aussi celles héritées de sa super-classe, et ainsi de suite jusqu'au plus haut niveau de la hiérarchie. Bien sûr, si aucune méthode n'a ce nom, une erreur de compilation se produit.
- Le compilateur récupère les types statiques de tous les paramètres effectifs (les paramètres passés lors de l'appel à cette méthode, c'est à dire dans notre cas les types statiques de `param1`, `param2`, etc.). Il compare ces types à ceux des signatures de toutes les méthodes surchargées :
    - Si aucune signature ne correspond, une erreur de compilation se produit.
    - Si une seule signature correspond, cette signature est retenue.
    - Si, compte tenu des coercitions implicites possibles entre les types statiques des paramètres effectifs et les types des paramètres des signatures connues, plusieurs signatures correspondent, alors le compilateur détermine la signature de la méthode la *« plus spécifique »*

      La méthode la *« plus spécifique »* est celle dont les types des paramètres (dans la signature) sont les « plus proches » des types statiques utilisés lors de l'appel.

      Plus précisément : *une méthode est « plus spécifique » qu'une autre si tout appel de la première peut être remplacée par tout appel de la seconde sans provoquer d'erreur. La méthode de signature* `m(type1_1 p1, ..., type1_n pn)` *est donc « plus spécifique » que la méthode de signature* `m(type2_1 p2, ..., type2_n pn)` *si et seulement si pour tout i,* $1 \leq i \leq n$, `type1_i` *est une sous-classe de* `type2_i`.

En résumé, si il n'y a pas d'erreur de compilation, **le traitement à la compilation détermine la signature de la méthode qui sera *in fine* exécutée**, compte tenu :

- du type statique de l'objet appelant
- du type statique de chacun des paramètres effectifs utilisés lors de l'appel.

Si la méthode est une méthode de classe (déclarée `static`), la machine virtuelle exécutera directement la méthode de la classe `TypeStatique`. On parle alors de « liaison statique ».

Ce cas étant traité, on se préoccupe dans la suite du cas où la méthode est une méthode standard (qui n'est pas déclarée `static`).

Traitement à l'exécution (« liaison dynamique »)

Connaissant la signature précise de la méthode à exécuter, il s'agit maintenant de déterminer dans quelle classe figure la méthode qui sera effectivement exécutée, sachant que cette méthode peut être redéfinie (→ avec exactement la même signature) à divers niveaux de la hiérarchie de classes.

C'est la machine virtuelle qui s'en charge, à l'exécution.

Le mécanisme dit de « liaison dynamique » est le suivant :

- La machine virtuelle détermine le type dynamique de l'objet : `TypeDynamique` dans notre cas.
- Si la classe `TypeDynamique` définit une méthode ayant la signature recherchée, alors c'est cette méthode qui est exécutée.
- Sinon, on recommence en remontant la hiérarchie, c'est à dire en remplaçant `TypeDynamique` par sa super-classe.
- Et ainsi de suite, jusqu'à trouver la 1ère méthode qui a la signature attendue.

Notez qu'on est certain de finir par trouver la méthode qui a la signature qui convient, car `TypeDynamique` est plus bas que `TypeStatique` dans la hiérarchie de classes et qu'on est sûr que cette méthode existe au moins dans `TypeStatique` : cela a été vérifié à la compilation !

Enfin, même si la notion *d'abstraction* en POO sera vue dans la prochaine fiche pédagogique, au cas ou vous la connaîtriez déjà, remarquons qu'il est possible que notre méthode soit « abstraite » dans `TypeStatique`. Par contre, on est certain que cette méthode sera bien concrète dans la classe `TypeDynamique`. En effet, puisque l'objet a pu être alloué avec `new TypeDynamique(...)`, c'est que `TypeDynamique` est une classe concrète... et donc qu'une implantation est bien fournie quelque part dans la hiérarchie pour toutes les méthodes abstraites dont `TypeDynamique` a hérité !

Exemples d'appel de méthodes

L'exemple suivant démontre quelques appels de méthodes avec vos animaux préférés.

```
public class Test {     //1
    public static void main(String args[]) {     //2
        Animal tabFig[] = new Animal[2]; //3
        tabFig[0] = new PangolinALongueQueue("Kirikou", 124, 12);    //4
        tabFig[1] = new Prof("Esseur", 10); //5
```

```
        tabFig[0].setNbEcailles(1000);  //6
        tabFig[0].setNom("Hubert"); //7
        System.out.println( tabFig[0].toString() ); //8

        tabFig[0].crier();  //10
        tabFig[1].crier("Les élèves dorment !");    //11
    }   //12
}   //13
```

Voici quelques explications :

- **Ligne 6. Le compilateur génère une erreur de compilation.** L'objet `tabFig[0]`, de type dynamique `PangolinALongueQueue`, possède bien une méthode `void setNbEcailles(int nb)` (cette méthode est héritée de `Pangolin`)... Mais le compilateur ne voit pas cette méthode ! En effet :
  - le type statique de `tabFig[0]` est `Animal`
  - la méthode `void setNbEcailles(int)` n'est pas définie au niveau de la classe `Animal`
  - le compilateur ne trouve donc pas de méthode correspondant à l'appel.

  Cela peut sembler ennuyeux, mais c'est en fait un gage de sécurité : le compilateur s'assure que l'appel de méthode pourra bien être exécutée par la machine virtuelle. On retiendra que :

  Pour pouvoir appeler une méthode sur un objet, il faut que cette méthode soit déclarée dans la classe du type statique de la référence utilisée pour le manipuler.

- **Ligne 7. La méthode exécutée est la méthode `void setNom(String)` définie dans la classe `Animal`.** En effet :
  - À la compilation, la signature de la méthode la « plus spécifique » trouvée dans la classe `Animal` est `void setNom(String)`.
  - Comme cette méthode n'est pas redéfinie dans les sous-classe, c'est elle qui est exécutée.

  L'attribut `nom` de notre pangolin, hérité dans la classe `PangolinALongueQueue`, est donc bien modifié. Bonjour Hubert ! - **Ligne 8. La méthode exécutée est la méthode `String toString()` définie dans `PangolinALongueQueue`.** En effet : - A la compilation, la signature de la méthode la plus spécifique trouvée dans `Animal`, est `String toString()`. - A l'exécution, c'est la version de la sous-classe `PangolinALongueQueue` qui est exécutée, puisque la méthode est redéfinie dans cette classe.

  Rappel en passant : notez que le code de cette méthode réalise un appel explicite à la version de la super-classe. - **Ligne 10. La méthode exécutée est la méthode `void crier()` définie dans la classe `PangolinALongueQueue`** (mêmes raisons que précédemment). - **Ligne 11. La méthode exécutée est la méthode `void`**

`crier(String raison)` **de la classe** `Animal`. En effet : - A la compilation, la signature de la méthode la plus spécifique trouvée dans `Animal`, est `void crier(String raison)`. - Cette méthode n'est pas redéfinie dans les sous-classes. C'est donc cette méthode que possède l'objet `Prof`, et c'est elle qui est exécutée.

Notez que cette méthode `String crier(String raison)` fait appel à `String crier()` (sans paramètre). Pour cet appel : - A la compilation, le type de `this` (type statique) est `Animal`, puisqu'on est dans cette classe. La classe `Animal` possède bien une méthode `void crier()` sans paramètre. Tout va bien... - A l'exécution, le type dynamique de l'objet étant `Prof`, c'est la version de `void crier()` définie dans la classe `Prof` qui est exécutée.

In fine, nous obtiendrons quelque chose comme : `"Les élèves dorment ! Du coup je crie : grrrrrrrr"`.

Exemple : le cas de la méthode equals

Votre professeur préféré vous a sans doute déjà indiqué que, en Java, lorsqu'on veut pouvoir tester l'égalité sémantique entre deux objets (l'égalité de l'état de ces objets) il faut écrire dans la classe une méthode de signature `public boolean equals(Object o)`.

Il faut que cette méthode prenne *toujours* bien un `Object` en paramètre, et pas autre chose. En effet, il s'agit de **redéfinir la méthode** `public boolean equals(Object o)` **de la classe** `Object` – et non pas de *surcharger* cette méthode. Il faut donc respecter la signature de la méthode d'origine.

Mais pourquoi cela ? On peut le comprendre maintenant que l'on connaît mieux le mécanisme d'appel de méthodes et la liaison dynamique.

Considérons deux versions d'une classe `Position`, la première avec une erreur dans la méthode `equals`, la seconde sans erreur.

```
class PositionErreur {
    public int x;
    public int y;
    ...

    // Ceci est une SURCHARGE de la méthode
    //       boolean equals(Object o)
    // héritée de la classe Object
    public boolean equals(PositionErreur other) {
        return other.x == this.x && other.y == this.y;
    }
}
class PositionOK {
    public int x;
    public int y;
    ...
    // Ceci est une REDEFINITION de la méthode
    //       boolean equals(Object o)
    // de la classe Object
    public boolean equals(Object o) {
```

```
        if( ! (o instanceof PositionOk)) {
                return false ;
        }
        // ou, plus restrictif :
        // if(o.getClass() != this.getClass()) {
        //  return false ;
        //}

        // cast explicite nécessaire, car Downcast :
        PositionOk other = (PositionOk) o;
        return other.x == this.x && other.y == this.y;
    }
}
```

Considérons maintenant le code de test suivant :

```
public class Test { // 1
   public static void main(String args[]) { // 2
      PositionErreur posErr1 = new PositionErreur( 1, 1 ) ; // 3
      PositionErreur posErr2 = new PositionErreur( 1, 1 ) ; // 4
      // tout ces casts sont licites, puisque toute classe java
      // (et donc en particulier PositionErreur)
      // dérive de la classe Object
      // (éventuellement via plusieurs niveau d'héritage)
      Object objErr1 = posErr1; // 9
      Object objErr2 = posErr2; // 10

      System.out.println("posErr1 eq posErr2 : " +
posErr1.equals(posErr2)); // 12
      System.out.println("posErr1 eq objErr2 : " +
posErr1.equals(objErr2)); // 13
      System.out.println("objErr1 eq posErr2 : " +
objErr1.equals(posErr2)); // 14
      System.out.println("objErr1 eq objErr2 : " +
objErr1.equals(objErr2)); // 15

      PositionOK posOk1 = new PositionOK( 2, 2 ) ; // 17
      PositionOK posOk2 = new PositionOK( 2, 2 ) ; // 18

      Object objOk1 = posOk1; // 20
      Object objOk1 = posOk2; // 21

      System.out.println("posOk1 eq posOk2 : " + posOk1.equals(posOk2)); //
22
      System.out.println("posOk1 eq objOk2 : " + posOk1.equals(objOk2)); //
23
      System.out.println("objOk1 eq posOk2 : " + objOk1.equals(posOk2)); //
24
      System.out.println("objOk1 eq objOk2 : " + objOk1.equals(objOk2)); //
25
   }
}
```

A priori, on voudrait que ce programme affiche 8 fois `"true"`, puisque les objets `posErr1` et `posErr2` sont « égaux » et que les objets `posOk1` et `posOk2` sont aussi « égaux ».

Or voici ce qui se passe...

- **Ligne 12. Tout se passe bien et il s'affiche `"posErr1 eq posErr2 : true"`.**
- **Ligne 13 : il s'affiche `"posErr1 eq objErr2 : false"`.** En effet :
  - le type statique de l'objet `posErr1` sur lequel la méthode est appelée est `PositionErreur`.
  - le type statique du paramètre `objErr2` est `Object`.

    - dans la classe `PositionErreur`, la *méthode la plus spécifique* a pour signature `boolean equals(Object)`. Il s'agit de la méthode héritée de la classe `Object`.
    - c'est donc la méthode `boolean equals(Object)` de la classe `Object` qui est exécutée.
    - cette méthode compare l'égalité des références (des adresses en mémoire). Elle renvoie donc `false`, puisque les deux objets n'ont pas la même adresse !
- **Ligne 14 : il s'affiche `"objErr1 eq posErr2 : false"`.** En effet :
    - le type statique de l'objet `objErr1` sur lequel la méthode est appelée est `Object`.
    - le type statique du paramètre `posErr2` est `PositionErreur`.
    - dans la classe `Object`, la *méthode la plus spécifique* a pour signature `boolean equals(Object)` (moyennant un cast implicite de `PositionErreur` vers `Object` pour le paramètre `posErr2`).
    - c'est donc la méthode `boolean equals(Object)` de la classe `Object` qui est exécutée.
    - cette méthode compare l'égalité des références (des adresses en mémoire). Elle renvoie donc `false`, puisque les deux objets n'ont pas la même adresse.
- **Ligne 15 : il s'affiche `"objErr1 eq objErr2 : false"`.** En effet :
    - le type statique de l'objet `objErr1` sur lequel la méthode est appelée est `Object`.
    - le type statique du paramètre `objErr2` est `Object`.
    - dans la classe `Object`, la *méthode la plus spécifique* a pour signature `boolean equals(Object)`.
    - c'est donc la méthode `boolean equals(Object)` de la classe `Object` qui est exécutée.
    - cette méthode compare l'égalité des références (des adresses en mémoire). Elle renvoie donc `false`, puisque les deux objets n'ont pas la même adresse.
- **Ligne 22 à 25 : dans tous les cas, c'est bien `"true"` qui s'affiche**, comme attendu. En effet, la seule méthode nommée `equals` de la classe `PositionOK` a pour signature `boolean equals(Object)` et elle est en accord avec tous les types statiques utilisés. C'est donc cette méthode qui sera exécutée.

Principe

En java, la méthode qui teste l'égalité de deux objets doit *toujours* avoir pour signature `public boolean equals(Object o)`.